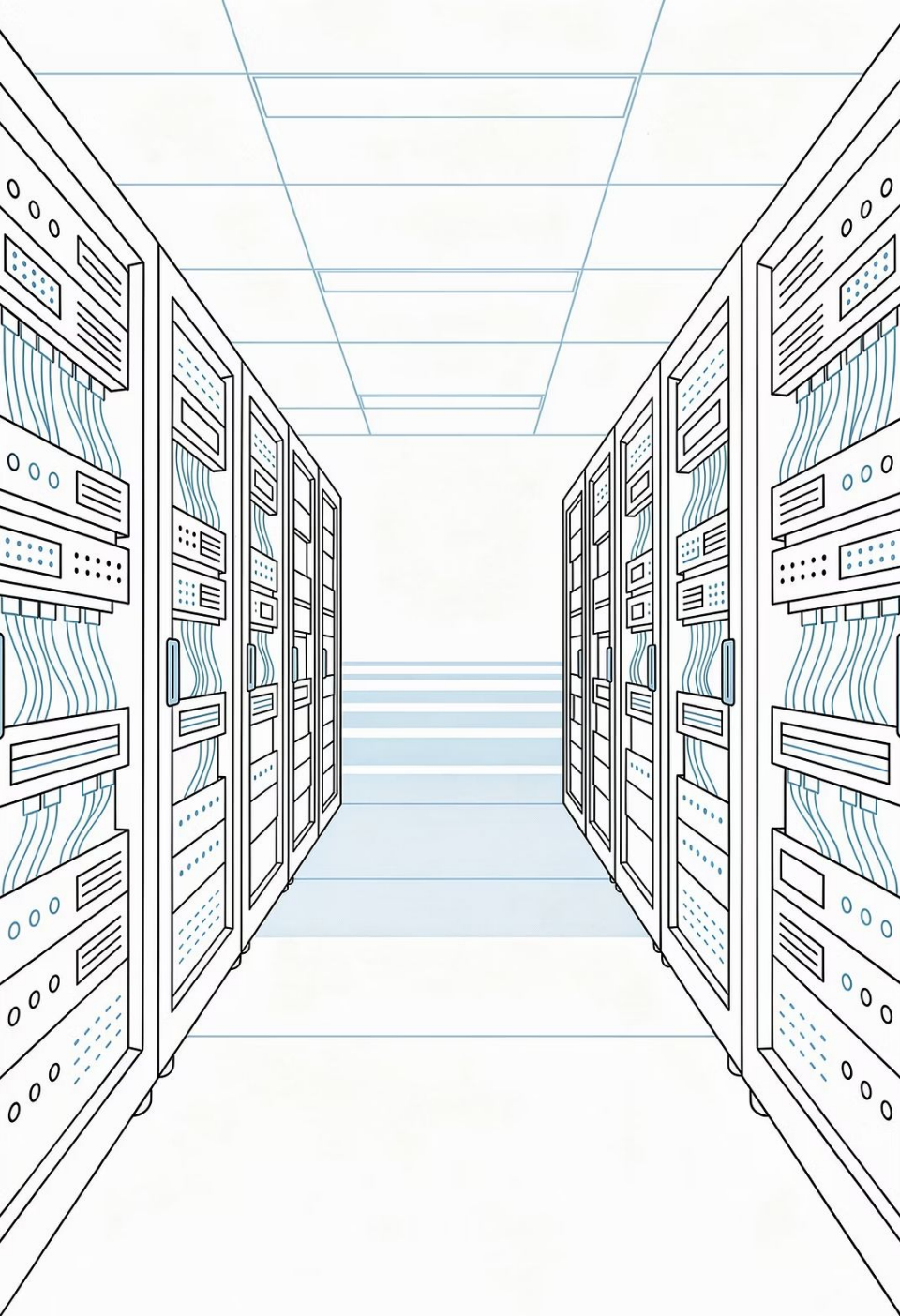




Prática de Controle de Concorrência



Eduardo Ogasawara

eogasawara@cefet-rj.br | <https://eic.cefet-rj.br/~eogasawara>



Compreender Concorrência

Entender como o PostgreSQL gerencia transações simultâneas e mantém a consistência dos dados

Observar Comportamentos

Analisar MVCC, locks explícitos, níveis de isolamento e transações concorrentes em ação

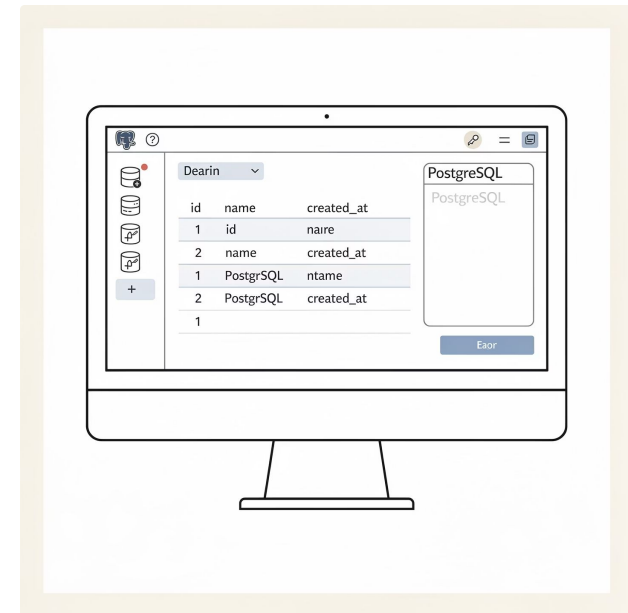
Relacionar Teoria e Prática

Conectar os conceitos teóricos vistos em aula com exemplos práticos e observáveis

Configuração Necessária

Para realizar os experimentos, você precisará configurar um ambiente com múltiplas sessões simultâneas conectadas ao PostgreSQL. Cada sessão representará uma transação independente, permitindo observar o comportamento do controle de concorrência.

- SGBD: PostgreSQL
- Ferramentas: pgAdmin, psql ou DBeaver
- Abrir 2 ou 3 sessões simultâneas
- Todas conectadas ao mesmo banco de dados



Criar Tabelas

Execute o script Postgres-CreateTables.sql para criar o esquema de exemplo com todas as tabelas necessárias

Popular Dados

Execute o script Postgres-Inserts.sql para inserir os dados iniciais nas tabelas criadas

Confirmar Dados

Verifique que a tabela EMPREGADO contém os dados iniciais corretamente inseridos

```
-- Verificar dados iniciais  
SELECT * FROM EMPREGADO;
```

O PostgreSQL utiliza MVCC (Multi-Version Concurrency Control), um mecanismo sofisticado que permite alta concorrência sem comprometer a consistência dos dados. Este sistema cria versões múltiplas dos dados para diferentes transações.

Leituras Não Bloqueiam Escritas

Transações de leitura podem acessar dados enquanto outras transações estão modificando-os

Escritas Não Bloqueiam Leituras

Transações de escrita não impedem outras transações de ler os dados

Dirty Reads Não Permitidos

Transações nunca veem dados não confirmados de outras transações

Bloqueios Específicos

Locks aparecem apenas em casos específicos de conflito de escrita

O PostgreSQL oferece diferentes níveis de isolamento para controlar como as transações interagem entre si. Cada nível oferece um equilíbrio diferente entre desempenho e consistência.

READ COMMITTED

Nível padrão do PostgreSQL. Lê apenas dados já confirmados.

REPEATABLE READ

Garante leituras consistentes durante toda a transação.

SERIALIZABLE

Maior nível de isolamento, garante equivalência serial.



Observação importante: READ UNCOMMITTED é tratado como READ COMMITTED no PostgreSQL.

```
SHOW default_transaction_isolation;
```

Observando o MVCC em Ação

Sessão 1

```
BEGIN;  
  
INSERT INTO EMPREGADO  
VALUES ('John','E','Borg',0,  
       '1929-11-10',  
       'Wall Street','M',  
       55000,NULL,1);
```

Transação iniciada mas não confirmada

Sessão 2

```
SELECT * FROM EMPREGADO;
```

Perguntas:

- O novo registro aparece?
- Por quê?

Sessão 2 Não Vê o Registro

A inserção não aparece na consulta da Sessão 2

Transação Não Confirmada

A Sessão 1 ainda não executou COMMIT

Visão Consistente

PostgreSQL fornece snapshot consistente dos dados

MVCC Evita Dirty Reads

Proteção automática contra leituras sujas



Importante: Este comportamento garante que cada transação veja uma versão consistente do banco de dados, mesmo com múltiplas transações simultâneas modificando os dados.

Encerrando a Transação da Sessão 1

Antes de seguir para o próximo experimento, confirme o INSERT que ficou pendente.

```
-- Sessão 1  
COMMIT;  
  
-- Sessão 2  
SELECT * FROM EMPREGADO  
WHERE CPF = '0';
```

1 Por que encerrar?

Sem COMMIT, o próximo BEGIN da Sessão 1 é ignorado (WARNING: there is already a transaction in progress) e os INSERTs dos dois experimentos seriam confirmados juntos.

2 O que observar

Após o COMMIT, a Sessão 2 passa a ver John Borg: dados confirmados tornam-se visíveis para as demais transações.

3 Para depois

O registro CPF = '0' será usado no experimento de bloqueio com SELECT ... FOR UPDATE.

O PostgreSQL oferece views do sistema que permitem observar o estado das transações em tempo real. A view `pg_stat_activity` é essencial para diagnóstico e monitoramento.

```
SELECT pid, state, wait_event_type,  
       wait_event, query  
FROM pg_stat_activity  
WHERE datname = current_database();
```

Sessões Ativas

Identificar quais conexões estão atualmente ativas no banco de dados

Sessões em Espera

Detectar transações que estão aguardando recursos ou locks

Consultas em Execução

Visualizar os comandos SQL executados em cada sessão

Consulta de Locks

```
SELECT pid, locktype,  
       mode, granted,  
       relation::regclass  
FROM pg_locks;
```

Esta consulta revela informações detalhadas sobre todos os locks ativos no sistema.

Informações Obtidas

Tipo de Lock

Identifica se é lock de tabela, linha, transação, etc.

Modo do Lock

Indica o nível de restrição (compartilhado, exclusivo, etc.)

Status de Concessão

Mostra se o lock foi concedido ou está aguardando

O nível de isolamento READ COMMITTED é o padrão no PostgreSQL. Vamos observar como ele se comporta com transações concorrentes.

Sessão 2: Inicia Transação

```
BEGIN;  
SELECT COUNT(*) FROM  
EMPREGADO;
```

Sessão 1: Insere e Confirma

```
BEGIN;  
INSERT INTO EMPREGADO  
VALUES ('John','E','Borg',1,  
        '1930-01-01',  
        'Wall Street','M',  
        60000,NULL,1);  
COMMIT;
```

Sessão 2: Nova Consulta

```
SELECT COUNT(*) FROM  
EMPREGADO;
```

O que acontece com o resultado?

Comportamento do Snapshot

No nível READ COMMITTED, cada comando SQL usa um snapshot atualizado do banco de dados. Isso significa que a transação pode ver diferentes versões dos dados ao longo de sua execução.

- Cada comando usa um snapshot atualizado
- Não vê mudanças não confirmadas
- Pode ver mudanças confirmadas entre comandos
- READ COMMITTED é o padrão no PostgreSQL



Este comportamento permite maior concorrência, mas pode resultar em leituras não repetíveis dentro da mesma transação.

O nível de isolamento REPEATABLE READ oferece maior consistência ao fixar o snapshot no início da transação.

Sessão 1: Define Isolamento

```
BEGIN ISOLATION LEVEL  
REPEATABLE READ;  
SELECT COUNT(*) FROM  
EMPREGADO;
```

Sessão 2: Insere Novo Registro

```
INSERT INTO EMPREGADO  
VALUES ('Jane','A','Doe','2','1931-02-  
02','Broadway','F',58000,NULL,1);  
-- autocommit: a inserção é  
confirmada imediatamente
```

Sessão 1: Consulta Novamente

```
SELECT COUNT(*) FROM  
EMPREGADO;  
COMMIT;
```

O resultado será o mesmo da primeira
consulta!

No PostgreSQL, o nível de isolamento deve ser definido dentro da transação (BEGIN ISOLATION LEVEL ... ou SET TRANSACTION logo após o BEGIN).

Snapshot Fixado

O snapshot é estabelecido no início da transação e permanece inalterado até o final, garantindo que todas as leituras vejam a mesma versão dos dados.

Isolamento de Mudanças

Mudanças confirmadas por outras transações não aparecem durante a execução da transação atual, mesmo após COMMIT de outras sessões.

Leitura Consistente

Garante que todas as consultas dentro da transação vejam exatamente os mesmos dados, evitando leituras não repetíveis e anomalias fantasma.

Sessão 1: Bloqueia Linha

```
BEGIN;  
  
SELECT * FROM EMPREGADO  
WHERE CPF = '0'  
FOR UPDATE;
```

A cláusula `FOR UPDATE` cria um lock explícito na linha selecionada, impedindo que outras transações a modifiquem.

Para liberar, execute COMMIT (ou ROLLBACK) na Sessão 1; o UPDATE da Sessão 2 prossegue imediatamente.

Sessão 2: Tenta Atualizar

```
UPDATE EMPREGADO  
SET SALARIO = 56000  
WHERE CPF = '0';
```

Resultado: A Sessão 2 bloqueia e fica aguardando a liberação do lock pela Sessão 1.

Quando duas sessões operam no nível SERIALIZABLE, o PostgreSQL monitora dependências de leitura/escrita para garantir a serializabilidade.

Sessão 1

```
BEGIN ISOLATION LEVEL SERIALIZABLE;  
SELECT SUM(SALARIO)  
FROM EMPREGADO  
WHERE DNO = 1;  
INSERT INTO EMPREGADO  
VALUES ('Ana','B','Lima','4',  
'1933-04-04','Main St',  
'F',50000,NULL,1);
```

Sessão 2

```
BEGIN ISOLATION LEVEL SERIALIZABLE;  
SELECT SUM(SALARIO)  
FROM EMPREGADO  
WHERE DNO = 1;  
INSERT INTO EMPREGADO  
VALUES ('Mark','T','Smith','3',  
'1932-03-03','5th Avenue',  
'M',59000,NULL,1);  
COMMIT;
```

Ordem: execute os dois SELECTs, depois os dois INSERTs, depois COMMIT na Sessão 2 e, por último, COMMIT na Sessão 1.

Quando a Sessão 1 executa COMMIT, o PostgreSQL detecta que cada transação leu dados que a outra alterou (nenhuma ordem serial explica os resultados) e aborta a Sessão 1 com:

could not serialize access due to read/write dependencies

Detecção

PostgreSQL detecta conflito de dependências

Abort

Transação é abortada para manter corretude

Reexecução

A transação deve ser reexecutada

O nível SERIALIZABLE oferece o mais alto grau de isolamento, mas com custos de desempenho associados.

Equivalência Serial

PostgreSQL garante que o resultado é equivalente a uma execução serial das transações.

Abort Controlado

Pode abortar transações estrategicamente para manter a correção dos dados.

Predicate Locks

Utiliza predicate locks para detectar conflitos entre transações concorrentes.

Trade-off

Maior custo de processamento em troca de maior isolamento e consistência.

É fundamental encerrar transações adequadamente e realizar manutenção periódica para evitar o acúmulo de versões antigas de dados.

Comandos Essenciais

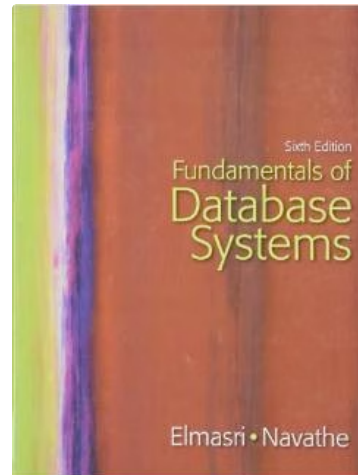
```
COMMIT;  
ROLLBACK;  
DELETE FROM EMPREGADO  
WHERE CPF IN ('0','1','2','3','4');  
VACUUM (ANALYZE);
```

O comando VACUUM é importante para recuperar espaço e atualizar estatísticas do banco de dados.

Por que é importante?

- Evita crescimento de versões antigas
- Melhora o desempenho
- Libera espaço em disco
- Atualiza estatísticas do otimizador

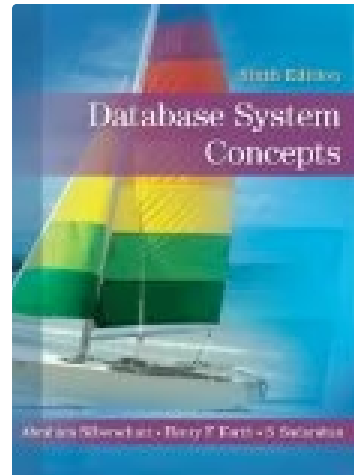
Referências



Elmasri & Navathe

Fundamentals of Database Systems

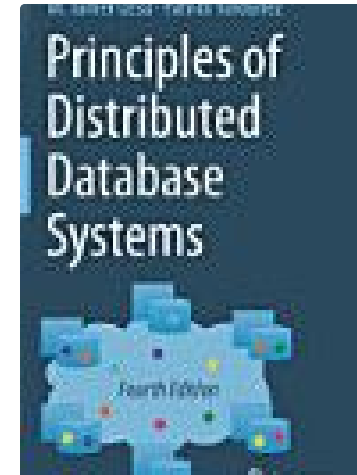
Pearson, 2016



Korth, Sudarshan & Silberschatz

Database System Concepts

McGraw-Hill, 2019



Özsu & Valduriez

Principles of Distributed Database Systems

Springer Nature, 2019